1、仿照04_Pydantic与tools实现文本翻译智能体

In [7]:
```
import os

from openai import OpenAI
from pydantic import BaseModel, Field

base_url = "https://dashscope.aliyuncs.com/compatible-mode/v1"
api_key = os.getenv("QWEN_API_KEY")
model_name = "qwen3.5-plus"
client = OpenAI(api_key=api_key, base_url=base_url)


# 文本翻译tool schema定义
class Translation(BaseModel):
    """对用户prompt进行翻译"""

    original_language: str = Field(description="原始语种")
    target_language: str = Field(description="目标语种")
    text: str = Field(description="待翻译文本")


# 定义文本翻译智能体
class Translator:
    def __init__(self, model_name: str):
        self.model_name = model_name

    def call(self, user_prompt: str, response_model: Translation):
        messages = [{"role": "user", "content": user_prompt}]
        tools = [
            {
                "type": "function",
                "function": {
                    "name": response_model.model_json_schema()["title"],
                    "dessciption": response_model.model_json_schema()["descripti
                    "parameters": {
                        "type": "object",
                        "properties": response_model.model_json_schema()["proper
                        "required": response_model.model_json_schema()["required
                    },
                },
            }
        ]
        response = client.chat.completions.create(
            model=model_name, messages=messages, tools=tools, tool_choice="auto"
        )
        message = response.choices[0].message
        if message.tool_calls:
            print("模型调用了工具")
            print(f"函数名:{message.tool_calls[0].function.name}")
            print(f"参数:{message.tool_calls[0].function.arguments}")
            arguments = message.tool_calls[0].function.arguments
            return response_model.model_validate_json(arguments)
        else:
            print("模型未调用工具")
            print(f"返回内容: {message.content}")
```

```
user_prompt = "请把以下内容翻译为德语：加载 ShellCode 经常要走常规的内存操作流程，
result = Translator(model_name).call(user_prompt, Translation)
```

模型调用了工具
函数名:Translation
参数:{"original_language": "中文", "target_language": "德语", "text": "加载 ShellCode 经常要走常规的内存操作流程，并且还需要解密 ShellCode，如果是自解密则需要 rwx，非自解密也会存在窗口期，BOF 也是如此。"}

2、04_goverment-advanced-rag部署

以主机上的vm虚拟机作为演示

1、系统环境管理

包管理器更新

```
sudo apt update
sudo apt upgrade -y
```

基础工具安装

```
sudo apt install -y git curl wget unzip vim net-tools lsof tree
build-essential
```

python安装

```
sudo apt install -y python3 python3-pip python3-venv
python3 --version
pip3 --version
```

创建虚拟环境+安装依赖

```
python3 -m venv <项目目录>/venv
source <项目目录>/venv/bin/activate
pip install -r <项目目录>/requirements.txt
```

2、elasticsearch

安装

```
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch |
sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-
keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/elasticsearch-
keyring.gpg] https://artifacts.elastic.co/packages/9.x/apt
stable main" | sudo tee /etc/apt/sources.list.d/elastic-9.x.list
sudo apt-get update
sudo apt-get install -y elasticsearch
```

还需要安装 IK 插件

```
curl http://localhost:9200 # 查看版本信息
sudo /usr/share/elasticsearch/bin/elasticsearch-plugin list # 查看有没有插件
sudo /usr/share/elasticsearch/bin/elasticsearch-plugin install https://release.infinilabs.com/analysis-ik/stable/elasticsearch-analysis-ik-9.3.1.zip
```

将elasticsearch跑起来

```
sudo /bin/systemctl daemon-reload
sudo /bin/systemctl enable elasticsearch.service
sudo systemctl start elasticsearch.service
sudo systemctl status elasticsearch.service
```

关闭elasticsearch的用户名+密码认证以及TLS认证，方便本地测试

编辑配置文件：

```
sudo nano /etc/elasticsearch/elasticsearch.yml
```

把和安全相关的配置改成或补成这样：

```
  discovery.type: single-node
xpack.security.enabled: false
xpack.security.http.ssl.enabled: false
xpack.security.transport.ssl.enabled: false
```

使用以下命令测试elasticsearch连接可用性

```
curl http://localhost:9200
```

3、config设置

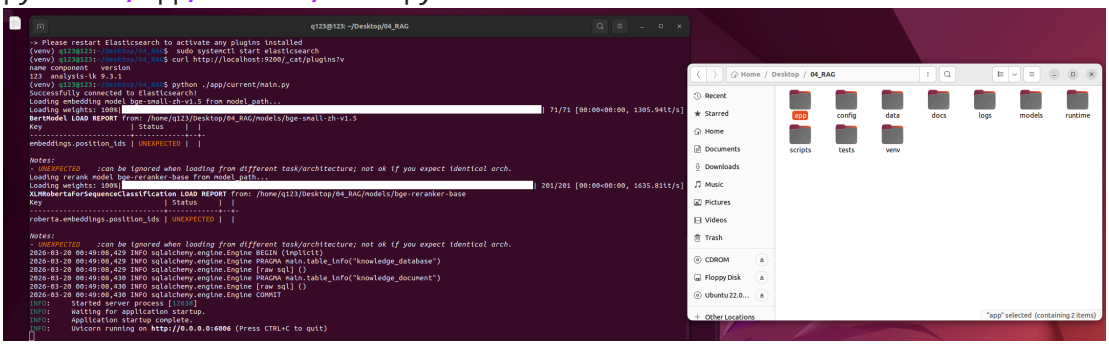
检查config设置以确保正确

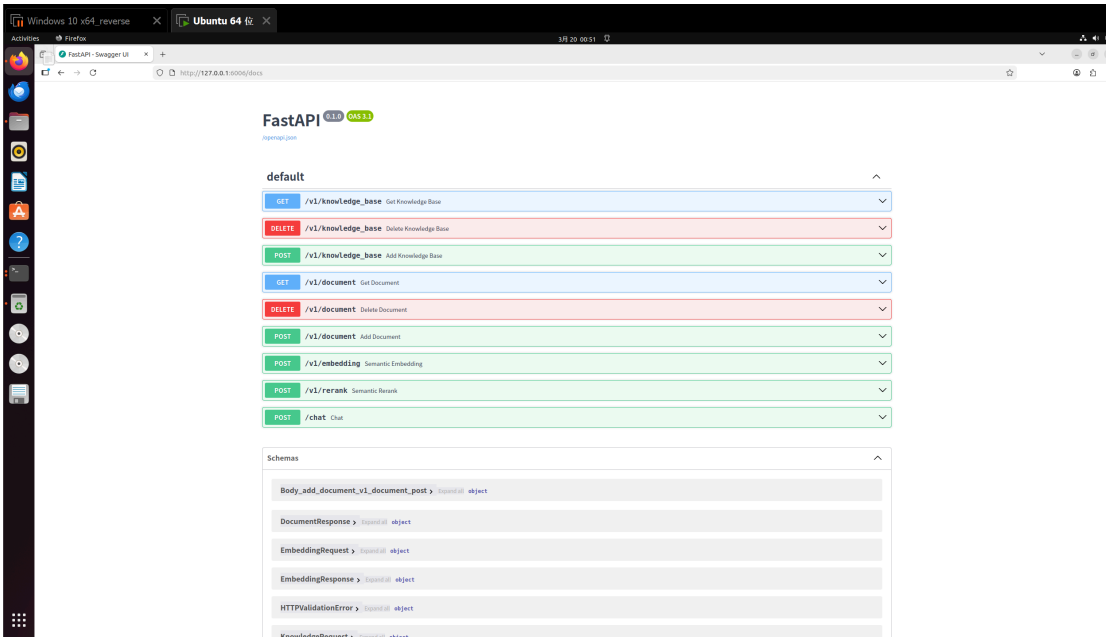
```yaml
elasticsearch:
  host: "localhost"
  port: 9200
  scheme: "http"
  username: ""
  password: ""

database:
  engine: "sqlite"
  path: "data/sqlite/rag.db"
  host: "localhost"
  port: 3306
  username: ""
  password: ""

models:
  embedding_model:
    bge-small-zh-v1.5:
      hf_url: "https://huggingface.co/BAAI/bge-small-zh-v1.5"
      local_url: "models/bge-small-zh-v1.5"
      dims: 512

  rerank_model:
    bge-reranker-base:
      hf_url: "https://huggingface.co/BAAI/bge-reranker-base"
      local_url: "models/bge-reranker-base"

  llm:
    config:
      temperature: 0.1
      top_p: 1
      max_tokens_to_sample: 1024
    base_model:
      glm:
        tokens: ""
      qwen:
        tokens: ""
      gpt:
        tokens: ""



device: "cpu"


storage:
  upload_dir: "data/uploads"



rag:
  llm_base: "https://dashscope.aliyuncs.com/compatible-mode/v1"
  llm_api_key: "sk"
  llm_model: "qwen3.5-plus"
  embedding_model: "bge-small-zh-v1.5"
  rerank_model: "bge-reranker-base"
  chunk_size: 256
  chunk_overlap: 20
  chunk_candidate: 10
  use_embedding: true
  use_rerank: true
  use_rrf: true
  port: 6006
```

4、运行并测试

该项目入口为main.py，直接使用以下命令即可

```
python ./app/current/main.py
```




Windows 10 x64_reverse
Ubuntu 64
Activities
Firefox
3月 20 00:51
FastAPI - Swagger UI
http://127.0.0.1:6006/docs
FastAPI
0.1.0
OAS 3.1
/openapi.json
default
GET /v1/knowledge_base Get Knowledge Base
DELETE /v1/knowledge_base Delete Knowledge Base
POST /v1/knowledge_base Add Knowledge Base
GET /v1/document Get Document
DELETE /v1/document Delete Document
POST /v1/document Add Document
POST /v1/embedding Semantic Embedding
POST /v1/rerank Semantic Rerank
POST /chat Chat
Schemas
Body_add_document_v1_document_post
DocumentResponse
EmbeddingRequest
EmbeddingResponse
HTTPValidationError

In [ ]: